Building LLM - General Project Report

Introduction

Building LLM is an educational project that begins with the smallest understandable language model and evolves one version at a time toward a modern small decoder-only LLM.

Each version introduces one focused concept while keeping the mechanisms visible, executable, and suitable for study.

```
one version -> one concept -> one verifiable result
```

Project goal

The goal is not to compete with industrial models. The goal is to understand how language models are represented, trained, evaluated, fine-tuned, and used, progressing from direct statistical counts to a decoder-only Transformer.

The project follows these principles:

- English is the initial modeling language;
- early mechanisms are implemented from first principles;
- complexity increases gradually and remains explainable;
- notebooks run from top to bottom;
- reproducibility matters more than model scale;
- advanced stages remain suitable for accessible Kaggle hardware.

Project constraints

- Version 1 uses standard Python only;
- NumPy is introduced before a machine-learning framework;
- TensorFlow appears only after a small neural model is understood;
- public datasets require documented origin, version, and license;
- each version keeps checks appropriate to its scope and avoids unnecessary production infrastructure.

What LLM means in this journey

The first versions are language models, but they are not yet large language models. The project uses the LLM label as an educational destination: a decoder-only Transformer with learned subword representations, public-data training, checkpointed model selection, controlled generation, instruction tuning, and multi-head attention.

The resulting model remains deliberately small. The name describes the architecture and learning journey rather than industrial parameter scale.

The stable language-model loop

Across the project, the conceptual loop remains stable:

```
text -> representation -> context
-> next-token probabilities
-> decoding -> generated text
```

Version 1 obtains probabilities directly from observed character counts. Later versions introduce trainable parameters, optimization, richer context, recurrent state, attention, subword tokenization, checkpointed training, held-out evaluation, supervised instruction tuning, and finally multiple causal attention heads.

Why the project evolves incrementally

The notebooks intentionally avoid replacing several mechanisms at once. A new version preserves the previous experiment whenever possible, so the effect of one new idea can be inspected directly. This keeps the project useful both as a learning sequence and as a technical portfolio: each notebook has a clear purpose, a limited scope, and a verifiable end state.

The real Version 1 foundation

Version 1 is a character-level statistical language model written with standard Python. It reads an embedded English corpus, creates adjacent character pairs, counts transitions, normalizes counts into probabilities, and samples one character at a time.

Its implemented checks verify the expected number of adjacent-character examples, probability normalization, reproducible generation with the same seed, and the requested generated length.

Version 1 has no trainable neural components. It establishes the language-modeling loop before optimization is introduced.

Roadmap - Statistical and neural foundations

Version 1 - Character statistical model

Counts adjacent character transitions, converts counts into probabilities, and generates text through reproducible sampling with standard Python.

Version 2 - Small neural network with NumPy

Replaces the count table with a small neural model and makes parameter learning visible through NumPy operations.

Version 3 - Training foundations

Introduces batches, data splits, a training objective, optimization, and systematic reproducibility.

Version 4 - TensorFlow introduction

Rebuilds the small model with TensorFlow and automatic differentiation after the underlying calculations are understood.

Version 5 - Embeddings and context windows

Learns vector representations and uses more than one preceding character as context.

Roadmap - Sequence modeling and attention

Version 6 - Recurrent language model

Introduces a simple RNN and a hidden state that carries information through a sequence.

Version 7 - GRU language model

Adds gated recurrent units so the sequence model can control how information is retained and updated.

Version 8 - Attention

Builds scaled dot-product attention from understandable components and examines how positions exchange information.

Version 9 - Transformer block

Combines causal self-attention, feed-forward layers, residual connections, and normalization.

Version 10 - Decoder-only Transformer

Stacks Transformer blocks and performs autoregressive generation with causal masking.

Design continuity

The transition from recurrence to attention does not discard the earlier lessons. Context, next-token objectives, probability distributions, gradient-based training, and reproducibility remain visible while the mechanism used to mix information across positions becomes progressively more expressive.

Roadmap - Public data, training, evaluation, and adaptation

Version 11 - Subword tokenization and public dataset

Moves from characters to a 2,048-token BPE vocabulary trained only on selected WikiText-103 training text, while preserving official train, validation, and test split boundaries.

Version 12 - Training pipeline and checkpoints

Makes training state explicit. LAST answers where training resumes; BEST answers which model validation selects. A clean simulated interruption restores LAST and continues to the 20-epoch budget before a fresh model restores BEST for final evaluation.

Version 13 - Evaluation and controlled generation

Keeps Version 12 training unchanged and adds held-out cross-entropy, perplexity, top-1/top-5 token accuracy, temperature, top-k, top-p, and reproducible autoregressive sampling.

Version 14 - Instruction tuning

Adds a small supervised fine-tuning stage using Databricks Dolly 15K. Explicit instruction/context/response formatting and a response mask restrict the supervised loss to response targets, while validation selects the best instruction checkpoint.

Version 15 - Multi-head Transformer

Preserves the complete Version 14 pipeline and splits the existing 16-dimensional attention space into two independent 8-dimensional causal heads. Projection shapes and the 34,496-parameter total remain unchanged, isolating multi-head attention as the latest architectural concept.

Dataset strategy

Version 1 embeds a tiny educational corpus directly in the notebook. Version 11 moves the project to WikiText-103 Raw v1, using deterministic controlled subsets while preserving the official split boundaries.

The BPE tokenizer is trained only on selected WikiText training text. Validation and test never influence vocabulary construction. Version 14 adds Databricks Dolly 15K as a separate instruction dataset; the existing BPE tokenizer is reused rather than retrained on instruction data.

For real datasets, the project records source, dataset identity, language, preprocessing choices, and the role of train, validation, and test data.

Kaggle strategy

- notebooks execute from top to bottom;
- seeds and key configuration values are explicit;
- external datasets are attached under /kaggle/input and missing inputs stop explicitly rather than silently falling back;
- checkpoint directories use /kaggle/working and are recreated for clean educational runs;
- model and dataset sizes remain conservative enough for accessible Kaggle hardware.

Quality and documentation

Each version ends with checks appropriate to its scope and a Notes section describing what changed, what stayed fixed, and what limitations remain. Later versions explicitly separate training, validation selection, and held-out test reporting. The project documents deliberate simplifications instead of presenting them as production features.

Current project milestone

The current project stage is an English decoder-only small LLM
and fine-tuned on accessible Kaggle hardware. Its current pipeline combines:

- a BPE tokenizer learned from public WikiText training text;
- fixed 16-token causal contexts and a two-block decoder-only Transformer;
- explicit LAST/BEST checkpoint semantics and resumable training inside a clean run;
- held-out evaluation with cross-entropy, perplexity, and token accuracy;
- controlled generation with temperature, top-k, top-p, greedy decoding, and reproducible sampling;
- supervised instruction tuning with response-focused masked loss;
- two-head causal self-attention while preserving the compact 34,496-parameter architecture.

The project remains intentionally small and transparent. It does not claim commercial-model scale, production serving infrastructure, RLHF, agents, or large-scale optimization. Its objective is a coherent, executable progression in which every version teaches one identifiable concept and closes with a verifiable result.

Current cycle summary

Versions 1 through 15 form a complete educational arc: statistical modeling -> neural learning -> sequence modeling -> attention -> decoder-only Transformer -> public subword data -> checkpointed training -> evaluation and controlled generation -> instruction tuning -> multi-head causal self-attention.

This current structure is consistent with the implemented notebooks and provides a clean portfolio narrative without adding complexity that is not demonstrated by the code.

The project remains open to future versions that can extend architecture, training, optimization, and alignment while preserving the same incremental learning approach.